KTester Replication Report

Replication of *Knowledge Matters: Injecting Project and Testing Knowledge into LLM-based Unit Test Generation* (Li et al., ICSE 2026; tool: KTester) using the authors' released code, data and result archive. Date: 2026-09-11. Full details: `ktester-replication-report-full.md`.

1. Replication question

Can the main experiment of the paper (Table 2, KTester row) be re-run from the released artifacts, and do the numbers it produces match the paper?

2. What was replicated

- Code: github.com/SYSUSELab/KTester (commit `fce83f1`), released prompts, static-analysis outputs and Lucene indexes.
- Dataset: the paper's 10 Java projects and 111 focal methods (the paper text says 110; the released data and the authors' own result archive both contain 111).
- Model: gpt-4o-mini, temperature 0.5, as stated in the paper. Calls routed through OpenRouter, pinned to OpenAI's endpoint.
- The authors' published results (`RQ1.7z`): 5 repetitions of the full experiment, used as the reference for comparison.

Two full runs were made, differing only in the repair budget: run A with `FIX_TRIES = 3` (the value in the released `settings.py.template`) and run B with `FIX_TRIES = 5` (the value stated in the paper).

3. Protocol

The released pipeline was run unchanged except for six small changes needed to execute on Linux (the authors developed on Windows): a missing settings class, shell invocation, classpath separators, a path-separator bug in the compile-error parser, an `lxml` dependency, and OpenRouter provider pinning. None of these touches prompts, model parameters or scoring. All runs happened in a Docker image (Java 17, Maven 3.9.9, Python 3.13) that is part of the archive. Every generated prompt, response, repair round, test class and coverage report is committed per project.

Prompts were compared with the authors' archived prompts for 30 tasks in 4 projects: the case-generation and code-generation prompts are identical apart from list ordering. The only difference found is a `@Timeout(600)` annotation present in the authors' initial-structure prompt and absent from the released template.

Metrics are computed from the tool's own `summary.json` with the same script for our runs and for the authors' archive, so both sides use identical definitions.

4. Results

4.1 The paper's numbers are recoverable from the authors' archive

Aggregating the authors' five archived repetitions reproduces Table 2 exactly for CPR (100%), LC (63.94%) and BC (55.46%). Two clarifications follow from this: the paper's "line coverage" is the instruction coverage field emitted by KTester's JaCoCo post-processing, and CPR/EPR are effectively test-case-level rates. EPR (paper 76.41%) is not exactly recoverable; the closest definition gives 78.81%. The number of tests per class is 8.06 in the archive versus 7.33 in the paper.

4.2 Our runs versus the authors' archive (identical definitions, 111 tasks)

	Compile pass (test cases)	Execution pass (test cases)	Instruction cov.	Branch cov.	Correct instr. cov.	Correct branch cov.	Tests / class	Time / task
Paper, Table 2	100.00	76.41	63.94	55.46	–	–	7.33	152.7 s
Authors' archive, mean of 5	100.00	78.81	63.94	55.46	55.52	47.21	8.06	152.7 s
Ours, run A (FIX_TRIES = 3)	82.19	51.80	47.77	43.40	37.59	33.95	10.52	84.0 s
Ours, run B (FIX_TRIES = 5)	85.71	47.25	44.27	39.30	32.26	27.90	10.47	150.0 s

"Correct" coverage counts only passing tests. Values are percentages. Final test classes that did not compile: 18 of 111 in run A, 16 of 111 in run B, 0 in every authors' repetition.

Per task, our correct instruction coverage falls inside the authors' five-repetition range for 80 of 111 tasks (run A) and 75 of 111 (run B); it is below the range for 24 and 33 tasks respectively. The gap is present in every project except commons-cli and is larger than the authors' own repetition-to-repetition spread in most of them.

5. Observations

1. The pipeline is executable and its behaviour matches the paper's description: the same prompts, the same three-perspective case generation, the same compile-execute-repair loop. The released code needed a Linux port but no algorithmic change.
2. The released configuration does not match the paper. `settings.py.template` sets 3 repair iterations; the paper says 5, and the authors' archive shows five rounds. Raising the budget to 5 recovered 10 of run A's 18 non-compiling classes but produced 8 new ones, raised compile pass by 3.5 points, and lowered execution pass and coverage. Run B's time per task (150 s) matches the paper's (153 s), which confirms 5 rounds is what the authors ran. The repair budget explains the time difference, not the quality gap.
3. With prompts, data, pipeline and configuration matched, our gpt-4o-mini runs still fall clearly short of the archive on every quality metric, consistently across projects and across the two runs. Our runs also produce more tests per class (10.5 vs 8.1). The authors' archive labels the KTester results `KTester-gpt4o`, while the paper says gpt-4o-mini. We cannot tell from the artifacts which model produced the archive. The observed gap is what one would expect if the archive came from a stronger model than the one named in the paper.
4. Where our runs fail, the failures are of the kind the repair loop cannot fix: the model invents types or members that do not exist in the project (13 of run A's 18 failures had their remaining error outside any `@Test` method, out of reach of KTester's rule that deletes failing test methods).

6. Limitations

- One run per configuration; the authors report five. Some of the run A / run B differences are within sampling noise.
- Two execution-environment effects lower our execution and coverage numbers but not compile rates: one task (`Sha2Crypt`) repeatedly hit the 10-minute test timeout, and jdom2 hit JVM out-of-memory errors in nine generated tests. The authors' `@Timeout(600)` annotation may have handled such cases in their runs.
- EPR (76.41 vs 78.81) and the task count (110 vs 111) could not be reconciled with the paper text.

7. Proposed next step

Run KTester on two or three projects with gpt-4o (not mini) and check whether the numbers land inside the authors' archive range. This directly tests the model-mismatch hypothesis at a cost of a few dollars.

Repository: branch `experiment/2026-09-week2-ktester-replication` in the KTester fork (handovers under `docs/`, artifacts under `results/`).